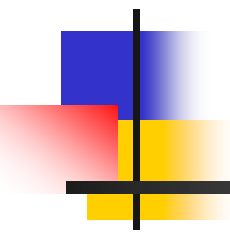
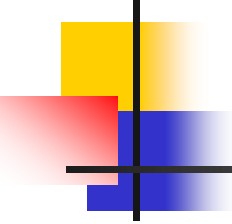


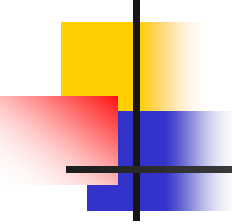
Forward and Reverse Engineering





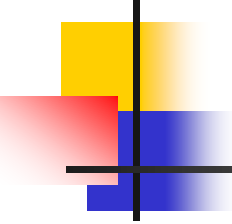
Forward and Reverse Engineering

- The UML is not just an OO modeling language. It also permits forward engineering (FE) and reverse engineering (RE).
- FE and RE [Som04] are engineering processes.
 - The objective of FE is to produce an implementation starting with a specification or a design.
 - The objective of RE is to recover a design or a specification from an implementation.



Forward and Reverse Engineering

- There are various CASE tools (e.g. Rational tools [IBM07]) which provide support for automatic FE and / or RE with the UML and various industrial-strength OO programming languages, such as Java or C++ [BB99,BB02,IBM07].
 - The support is limited but, in general, at least class diagrams can be handled automatically.
 - By combining automatic FE and RE, the UML models can be maintained in sync with the implementation at minimal costs.

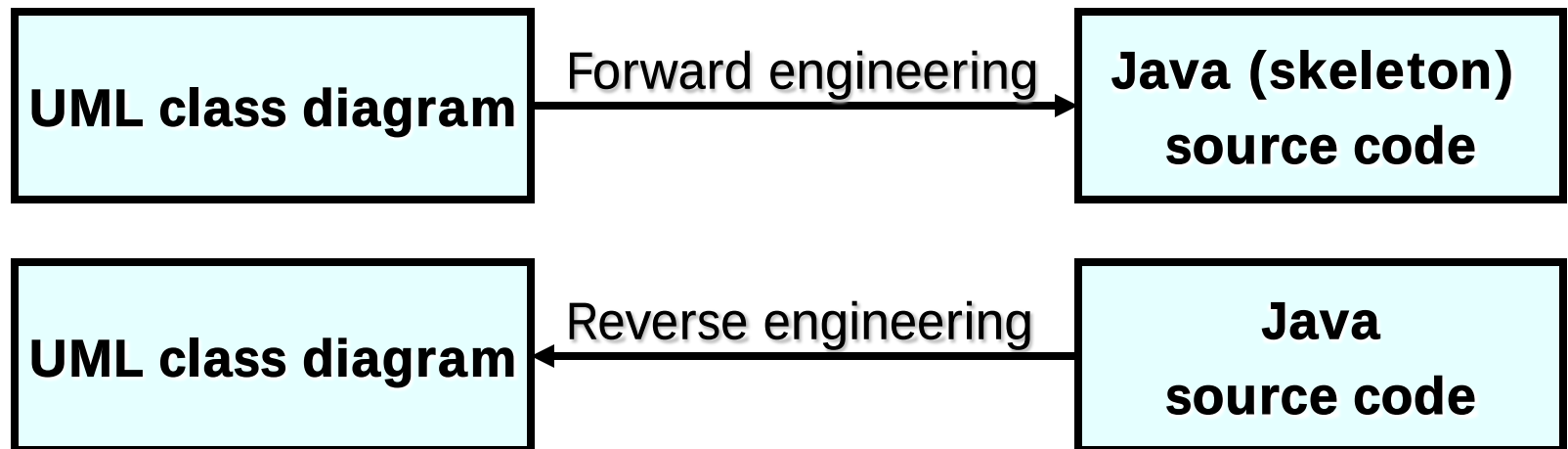


Forward and Reverse Engineering

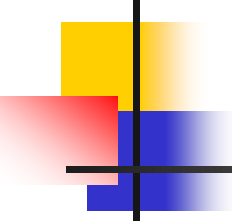
- In this section FE and RE experiments with UML & Java are presented. The presentation is essentially based on [BB99,BB02,BRJ99].
- Only class diagrams will be considered.
 - The experiments will help you to understand better
 - the semantics of UML class diagrams and
 - the relationship between UML class diagrams and an actual OO implementation.
 - This is important because class diagrams are the core of UML-based design models.

Forward and Reverse Engineering

- The FE and RE processes that we explain in this section are represented diagrammatically below.



- We begin with FE and then continue with RE.



Forward Engineering

UML → Java

Class

Class

- privateAttr : int
+ publicAttr : char
protectedAttr : long
packageAttr : double

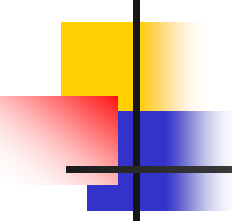
+ Class()
+ publicOp() : void
packageOp() : void

- UML model -

```
public class Class
{
    private int privateAttr;
    public char publicAttr;
    protected long protectedAttr;
    double packageAttr;

    public Class() {}
    public void publicOp() {}
    void packageOp() {}
}
```

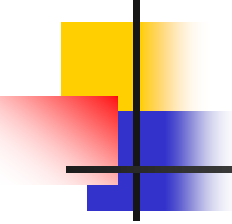
- Java code generated by FE -



Forward Engineering

UML → Java

- To support an **association relationship** attributes have to be generated in the code [BB99,BB02].
 - Role names can be used to specify association attributes (alternatively, association attribute names might be generated automatically by the FE tool).
 - Role names support visibility specifications (public,private,protected,package) but, for simplicity, in the sequel all attributes supporting associations will be **public**. We also assume that the FE tool automatically generates constructors.



Forward Engineering

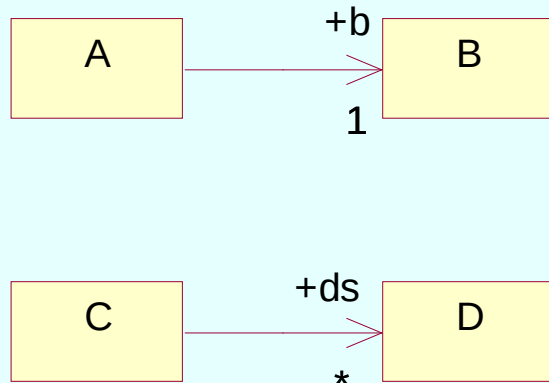
UML → Java

- In case of a unidirectional association from class A to class B, an attribute of type B must be generated in the body of class A.
- In case of a bi-directional association attributes must be generated in the both classes participating in the relationship.
- To support one-to-many or many-to-many relationships, arrays or other container classes are employed.

Forward Engineering

UML → Java

Unidirectional association



There are many (*) instances of class D for each instance of class C.

- UML model -

```
public class A {
    public B b;
    public A() {}
}
```

```
public class C {
    public D ds[];
    public C() {}
}
```

```
public class C {
    public Container ds;
    public C() {}
}
```

```
public class B {
    public B() {}
}

public class D {
    public D() {}
}
```

■ The FE tool might generate an array [BB02].

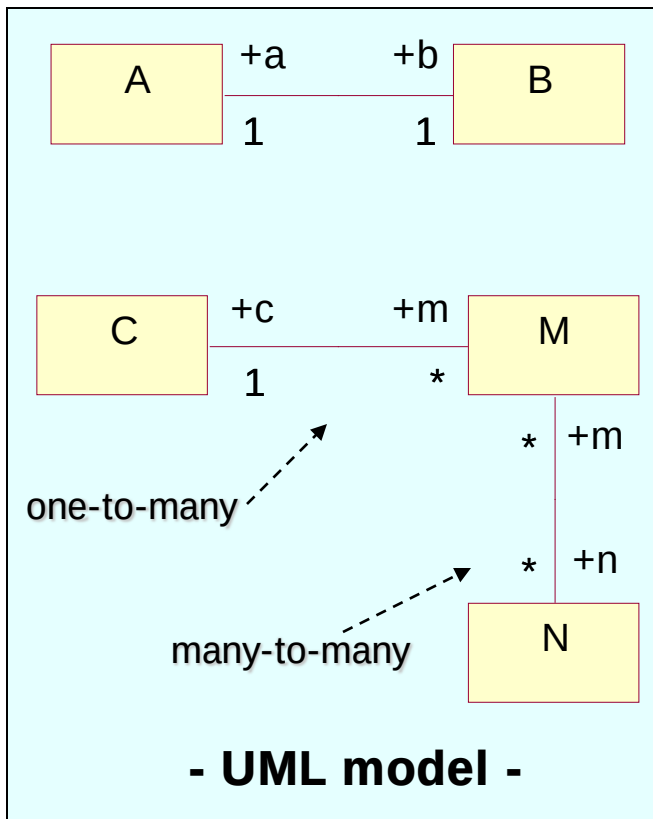
■ Alternatively, the FE tool might give you the option to specify a Container class.

- Java code generated by FE -

Forward Engineering

UML → Java

Bi-directional association



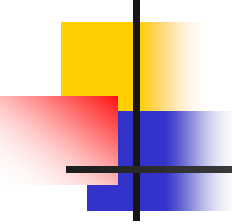
```
public class A {  
    public B b;  
    public A() {}  
}
```

```
public class C {  
    public M m[];  
    public C() {}  
}
```

```
public class B {  
    public A a;  
    public B() {}  
}
```

```
public class M {  
    public C c;  
    public N n[];  
    public M() {}  
}  
public class N {  
    public M m[];  
    public N() {}  
}
```

- Java code generated by FE -



Forward Engineering

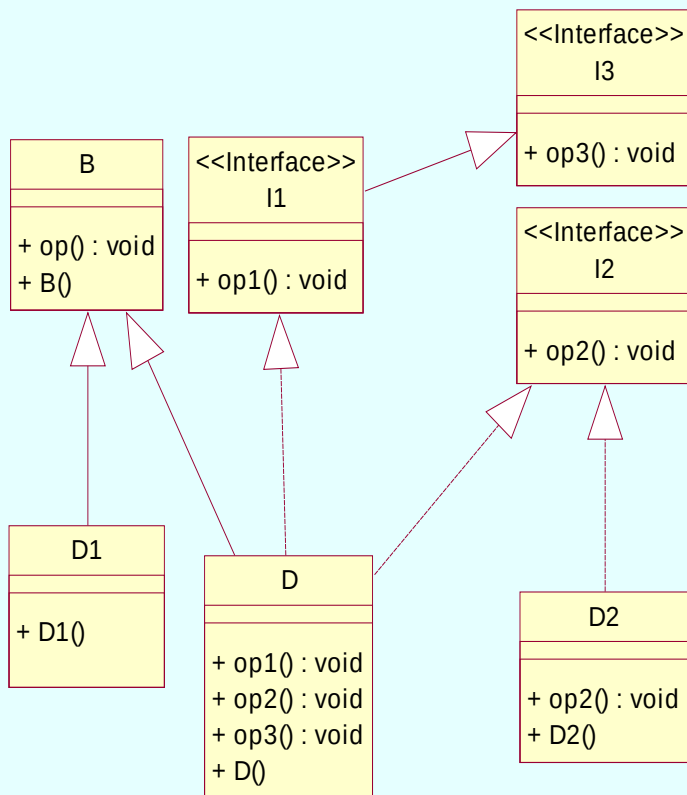
UML → Java

- **Generalizations** are implemented using the **extends** keyword in Java.
- In UML class diagrams, the **realization** relationship can be used to specify the relationship between an interface and a class that implements operations for it; in Java, this relationship is expressed using the **implements** keyword.

Forward Engineering

UML → Java

Generalization and realization



- UML model -

```
public interface I1 extends I3 {
    public void op1();
}

public class B {
    public B() {}
    public void op() {}
}

public class D1 extends B {
    public D1() {}
}

public class D extends B implements I1, I2 {
    public D() {}
    public void op1() {}
    public void op2() {}
    public void op3() {}
}

public interface I3 {
    public void op3();
}

public interface I2 {
    public void op2();
}

public class D2 implements I2 {
    public D2() {}
    public void op2() {}
}
```

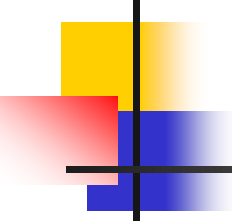
- Java code generated by FE -

Forward Engineering

UML → Java

- There are four kinds of relationships in the UML: dependency, association, generalization, and realization.
- The dependency is the most general relationship in UML diagrams.
- A **dependency** is a semantic relationship between two things in which a change to one thing (the independent thing) may affect the semantics of the other thing (the dependent thing) [BRJ99].
- A dependency is rendered as a dashed directed line, pointing to the thing being dependent on.





Forward Engineering

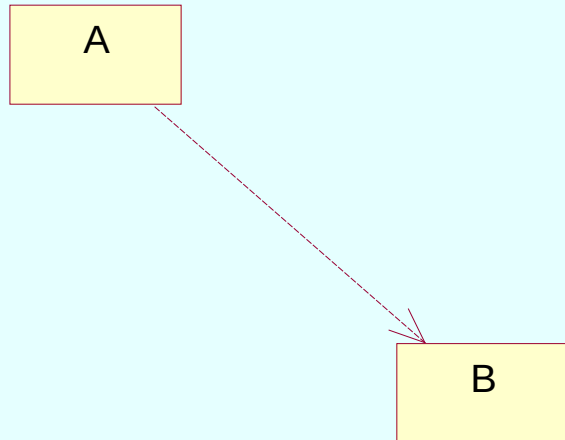
UML → Java

- You will use a **dependency** between class A and class B if [BB99,BB02]:
 - Class B is “global” (e.g., in Java, it has a public static attribute used by A)
 - Class A uses class B as an argument type in the signature of an operation.
 - Class B is instantiated as a local variable inside an operation of class A.

Forward Engineering

UML → Java

Dependency



- UML model -

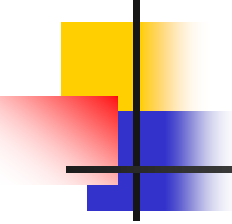
```
public class A {  
    public A() {}  
}
```

```
public class B {  
    public B() {}  
}
```

- The dependency is the most general relationship in UML diagrams.

- In this case the FE tool does not know what kind of dependency is assumed. Therefore, it generates no support for the relationship (this is the approach considered in [BB02]).

- Java code generated by FE -



Reverse Engineering

Java → UML

- UML models can be recovered from Java code by reverse engineering (RE).
 - Each Java class will be represented as a class in the UML model.
 - The relationships between classes can always be inferred from the source code.

Reverse Engineering Java → UML

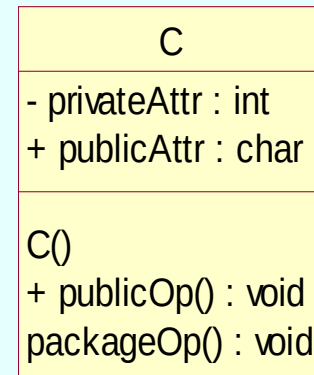
Classes and associations

```
public class C
{
    private int privateAttr;
    public char publicAttr;
    C() {}
    public void publicOp() {}
    void packageOp() {}
}
```

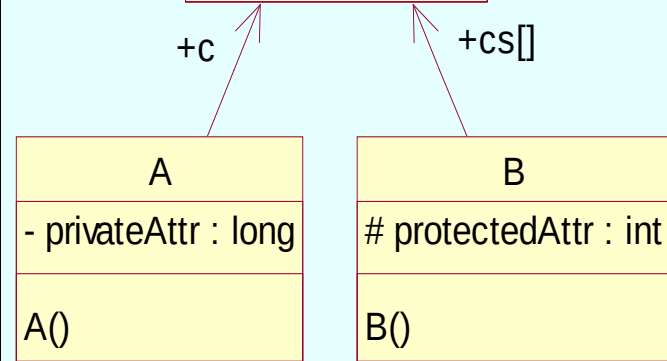
```
class A {
    private long privateAttr;
    public C c;
    A() {}
}

class B {
    protected int protectedAttr;
    public C[] cs;
    B() {}
}
```

- Java source code -



Each attribute will be represented as an association if its class is represented in the model.



- UML model generated by RE -

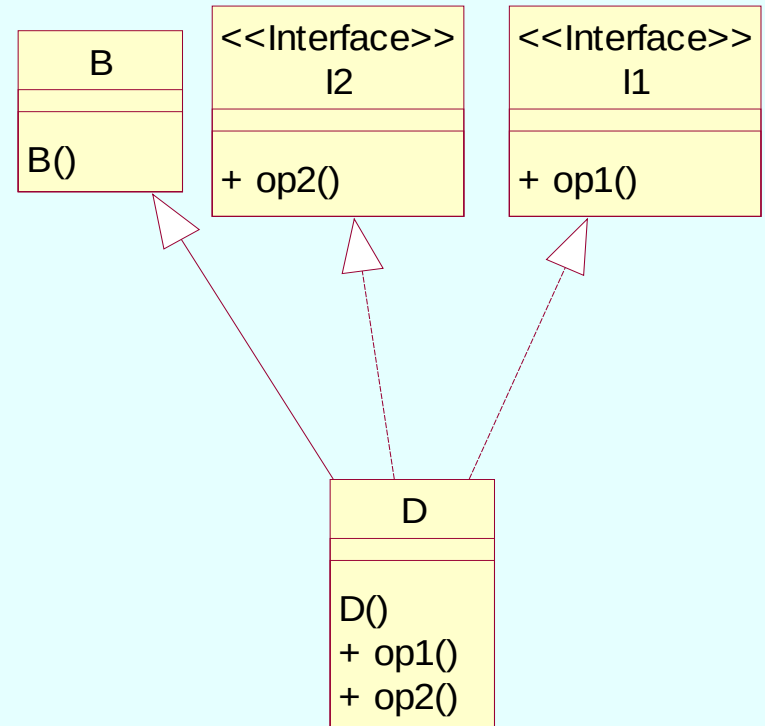
Reverse Engineering Java → UML

Generalization and realization

```
class B {  
    B() {}  
}  
  
interface I1 {  
    void op1();  
}  
  
interface I2 {  
    void op2();  
}
```

```
public class D extends B implements I1,I2 {  
    D() {}  
    public void op1() {}  
    public void op2() {}  
}
```

- Java source code -



- UML model generated by RE -

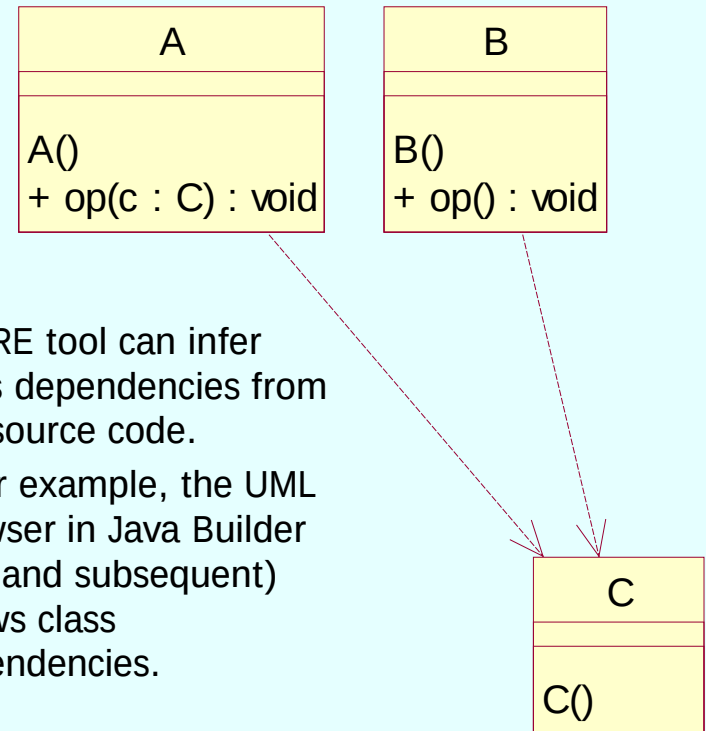
Reverse Engineering

Java → UML

Dependency

```
class A {  
    A() {}  
    public void op(C c) {}  
}  
  
class B {  
    B() {}  
    public void op() {  
        C c = new C();  
    }  
}  
  
public class C {  
    C() {}  
}
```

- Java source code -



- A RE tool can infer class dependencies from the source code.
- For example, the UML browser in Java Builder (6.0 and subsequent) shows class dependencies.

- UML model generated by RE -

Reverse Engineering Java → UML

A larger example

```

class L {
    private N head;
    public L() {...}
    public void empty() {...}
    public LI first() {...}
    public LI find(Object x) {...}
    public void insert(Object x, LI p) {...}
    public void remove(Object x) {...}
}

class LI {
    N current;
    LI(N n) {}
    public Object retrieve() {...}
    public void advance() {...}
}

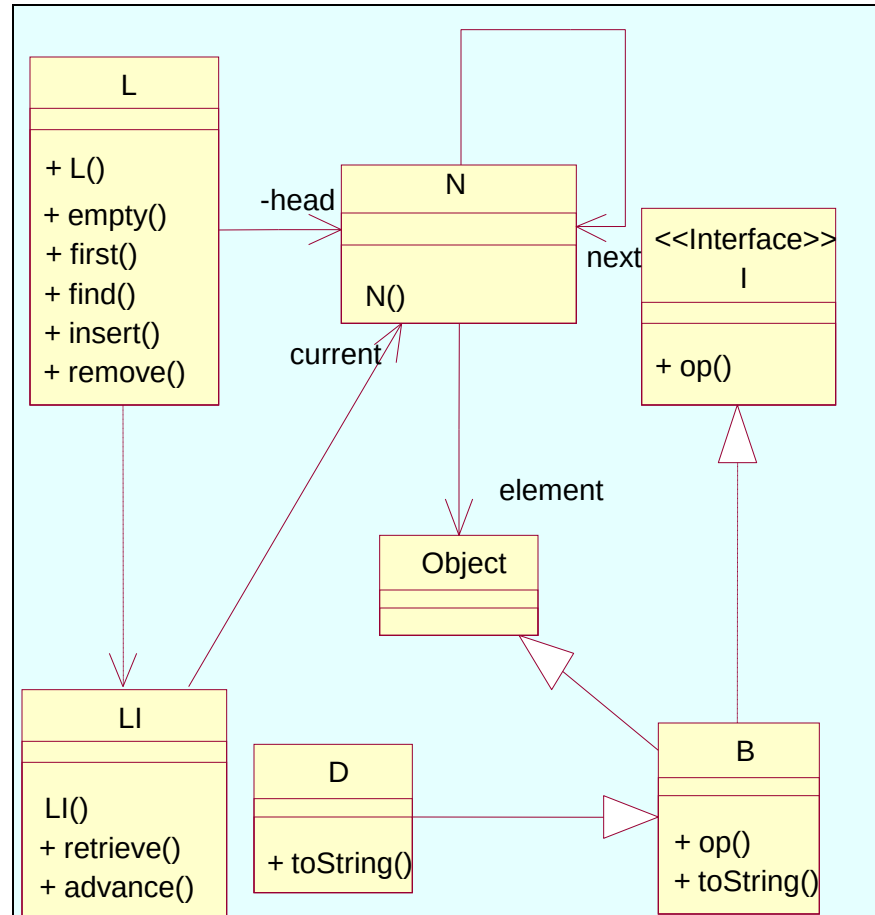
class N {
    Object element;
    N next;
    N(Object e, N n) {...}
}

interface I {
    void op();
}

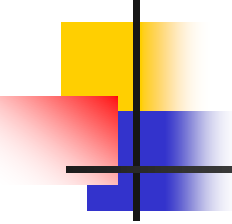
class B extends Object
    implements I {
    public void op() {}
    public String toString() {...}
}

class D extends B {
    public String toString() {...}
}
    
```

- Java source code -



- UML model generated by RE -



References

- [BB99] W. Boggs, M. Boggs. *Mastering UML with Rational Rose*. Sybex, 1999.
- [BB02] W. Boggs, M. Boggs. *Mastering UML with Rational Rose* (2nd edition). Sybex, 2002.
- [BRJ99] G. Booch, J. Rumbaugh, I. Jacobson. *The Unified Modeling Language User Guide*. Addison-Wesley, 1999.
- [Som04] J. Sommerville. *Software Engineering* (7th edition). Addison-Wesley, 2004.
- [IBM07] <http://www-306.ibm.com/software/rational/>